

Testing with pytest

Study notes · From assert to TDD, built on the Framingham risk calculator

Learning goals. After these notes you can (1) explain what a test is and why it matters in medical software, (2) write pytest tests with parametrize and pytest.raises, (3) choose good test data at the boundaries, (4) check that a test really tests something, and (5) run a full TDD cycle.

1. Why test?

A test is a **claim about your code that the computer checks for you**. Every test answers one question: *what must be true?*

Reason	In MediLink
Catch errors when code changes	If someone edits a coefficient, a reference test turns red
Document behaviour	The tests show what the function must do: treatment only above SBP 120
Prevent old bugs from returning	The mmol/L bug has its own test and cannot come back unnoticed
Required in medical software	The standard IEC 62304 requires documented verification for medical device software

2. From assert to pytest

Level	Tool	What you learned
1	assert	A test is a check: <code>assert result == expected</code>
2	pytest	Finds <code>test_*.py</code> files automatically, shows exactly what failed. Run: <code>py -m pytest -v</code>
3	parametrize, <code>pytest.raises</code>	One function, many cases; clean error tests

Parametrize: one function, many cases

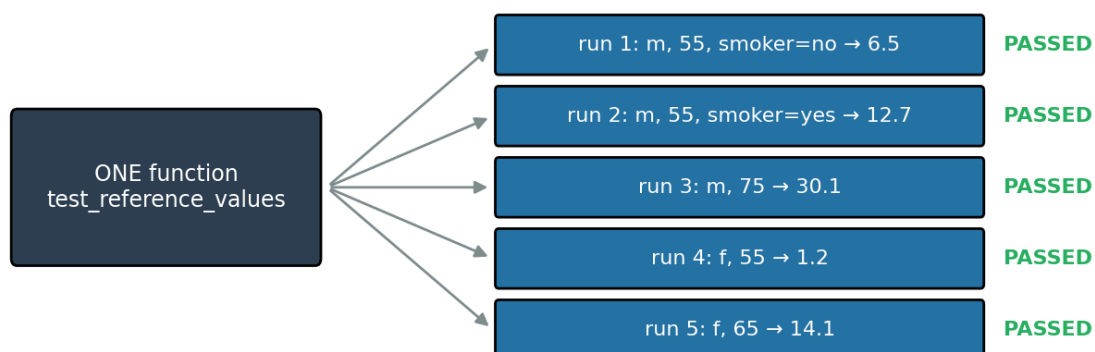


Figure 1 · pytest runs the same test once per case and names each run, e.g. `test_reference_values[args0-6.5]`.

Testing for errors: pytest.raises

"This call must fail" is written like this. If no ValueError is raised, the test fails with **DID NOT RAISE**.

```
with pytest.raises(ValueError):  
    framingham("m", 55, 5.5, 1.3, 120, False, False)
```

3. Good test data

Comparison tests: change exactly one thing

To test that treatment raises the risk, compare the **same patient** twice and change **only** the treatment flag. Then you know what caused the difference.

```
assert framingham("m", 50, 200, 50, 130, True, False) > \  
    framingham("m", 50, 200, 50, 130, False, False)
```

Boundaries: bugs live at the edges

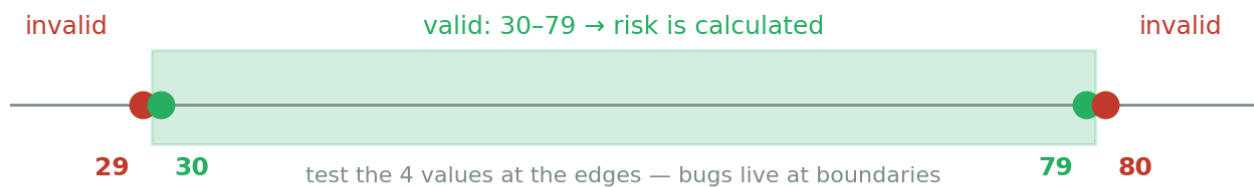


Figure 2 · For a valid range of 30-79, test 29 and 80 (must fail) and 30 and 79 (must work).

Input or output? Always be clear which one you are checking. 79 is a limit for the **age** (input); the **risk** (output) is a percentage between 0 and 100. Likewise, in the mmol/L test the values are cholesterol 5.5 and HDL 1.3 (inputs), not the factor 40 or the result 0.6 %.

4. Does your test really test something?

A test that checks nothing always passes. The TODO tests containing only pass were green. That is the most dangerous kind of test: it gives false confidence.

The break-it check: change the code or the check on purpose. A good test must turn red.

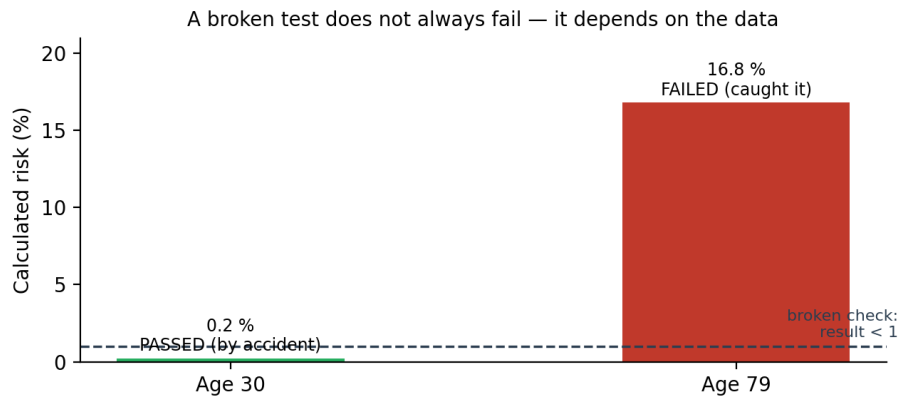


Figure 3 · With the broken check result < 1, age 30 passed by accident (0.2 %) and only age 79 caught the error (16.8 %). Testing both boundaries saved you.

You broke	Result	Lesson
Men's smoking age cap 70 → 80	The age-75 smoker reference test failed	The cap only matters for smokers above 70
result < 100 → result < 1	Age 30 passed, age 79 failed	A broken test does not always fail; it depends on the data

5. Test-driven development (TDD)

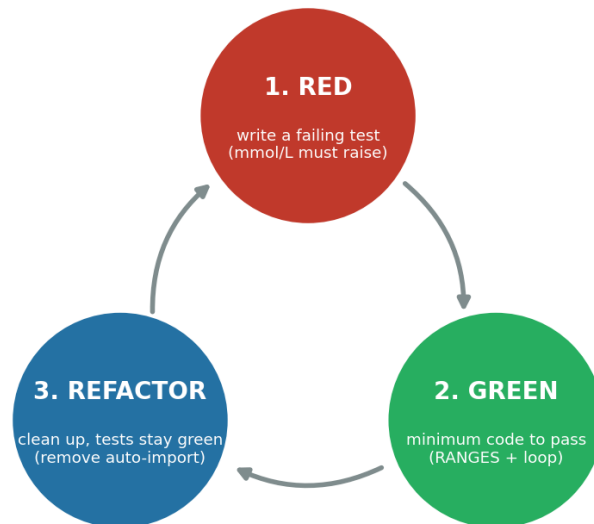


Figure 4 · The TDD cycle, with your mmol/L fix as the example.

Step	What you did	Why
Red	test_mmol_input_raises → DID NOT RAISE	Proves the bug exists and the test can detect it
Green	RANGES dictionary + loop in framingham()	Minimum code that makes the test pass
Refactor	Removed an auto-import, fixed a wrong comment	Clean code while all tests stay green

6. Pitfalls you met

Pitfall	What happened	Rule
Hard-coded value in a parametrized test	age=40 instead of the variable age: both runs tested 40	Use the parameter, never a fixed number
Missing test after deleting a file	test_mmol_input_raises vanished with test_risk_pytest.py	Count your tests: expected 16, got 15
Duplicate test files	31 tests instead of 16	One test file per module
VS Code auto-import	from unicodedata import name appeared	Check every import you didn't write
Outdated comment	Value 70, comment said 80	A wrong comment is worse than none

7. Commands

```
py -m pip install pytest      # install once
py -m pytest -v               # run all tests, verbose
py -m pytest test_risk.py -v  # run one file
git add risk.py test_risk.py
git commit -m "..."
```

```
git push
```

8. Self-test

Cover the right column and answer in your own words.

#	Question	Model answer
1	What is a test?	A claim about your code that the computer checks automatically.
2	Why is a test that checks nothing dangerous?	It always passes and gives false confidence that the code works.
3	What does <code>@pytest.mark.parametrize</code> do?	Runs one test function once per case, each run reported separately.
4	How do you test that a call raises an error?	with <code>pytest.raises(ValueError)</code> : followed by the call.
5	Which ages do you test for a valid range 30–79, and why?	29, 30, 79, 80: bugs are most likely at the boundaries.
6	Why must the test fail first in TDD?	It proves the bug exists and that the test can detect it; a test that never failed may check nothing.
7	Why did the broken age test pass at 30 but fail at 79?	Risk at 30 is 0.2 %, which is below 1 by accident; at 79 it is 16.8 %.
8	Why does testing matter more in medical software?	Wrong results can harm patients; IEC 62304 requires documented verification.

Question	1	2	3	4	5	6	7	8
First try								
Second try								